OBJEKTNO-ORIJENTISANO PROGRAMIRANJE · smjerovi B, C, D

# Od proceduralnog koda do OO koda koda

1. prelazak sa jezika C na C++
2. ograničenja strukturiranog programiranja
3. prva klasa

# Radno okruženje — Code::Blocks

1. File → New → Project → Console application → C++
2. Naziv projekta: OOP01_Racun; putanja **bez razmaka i bez naših slova**
3. Prevodilac: GNU GCC; konfiguracije Debug i Release ostaviti nepromijenjene
4. Settings → Compiler → Compiler Flags → označiti -std=c++17
5. F9 — Build and run;
6. Prazan projekat se prevodi i pokreće.
7. Sadržaj datoteke main.cpp obrisati u cjelosti

# Radno okruženje — VS Code

1. Napraviti folder bez razmaka i bez naših slova, npr. C:\OOPexamples\OOP01_Racun.
2. Otvoriti ga u VS Code: File → Open Folder.
3. Napraviti datoteku main.cpp („New File" u Explorer panelu) i napisati kod.
4. Run C/C++ code gore desno

# Koncepti naslijeđeni iz jezika C

Jezik C++ nastao je kao **proširenje** jezika C. Gotovo cijeli kod koji ste pisali u C prevodi se i kao C++ kod

### Osnova jezika

- Ugrađeni tipovi i operatori
- Naredbe kontrole toka
- Funkcije i rekurzija

### Memorija

- Pokazivači i adresna aritmetika
- Nizovi i lokalne promjenljive na steku (automatsko trajanje) – kompajler brine o oslobađanju memorije
- Dinamička alokacija – programer brine o oslobađanju memorije (postoji i u C++ ali je drugačije realizovana)

### Organizacija

- struct
- Razdvojeno prevođenje fajlova a potom linkovanje
- Preprocesor i zaglavlja

# Neke razlike koje uvodi C++

**nullptr**

Jezik C

```
int* p = NULL;
int  x = NULL;  /* kompajler neće prijaviti grešku*/
```

Jezik C++

```
int* p = nullptr;
int  x = nullptr;  // greska
```

NULL je cijeli broj 0 a nullptr koji uvodi C++ je pokazivač koji ne pokazuje ni na šta (najbolje je **ne korsititi NULL u C++)**

**auto**

```
double zbir = 0.0;
```

```
auto zbir = 0.0;  // kompajler sam zaključuje da je tip double
```

Kompajler sam izvodi tip iz inicijalizatora

**Tokovi**

bez format-specifikatora

```
printf("%s: %.2f\n", ime, x);
scanf("%lf", &x);
```

```
cout << ime << ": " << x << "\n";
cin >> x;
```

std::cout, std::cin omogućavaju razumijevanje tipova pri prevođenju, bez format-specifikatora poput  %d, %f, %s...

# Neke razlike koje uvodi C++

| | Jezik C | Jezik C++ |
|---|---|---|
| **std::string**<br>bez unaprijed zadate dužine | `char ime[30];`<br>`strcpy(ime, "Marko");`<br>`strcat(ime, " Markovic");`<br>podatak i njegova veličina su dvije odvojene stvari | `std::string ime = "Marko";`<br>`ime += " Markovic";`<br>podatak i njegova veličina su u istom objektu |

objekat std::string sam upravlja memorijom i dužinom

| | Jezik C | Jezik C++ |
|---|---|---|
| **Petlja po opsegu**<br>bez indeksa i bez granice | `for (int i = 0; i < n; i++)`<br>`  zbir += a[i];` | `for (double x : a)`<br>`  zbir += x;` |

nema indeksne aritmetike ni izlaska iz opsega

Ono što je u jeziku C bilo stvar pažnje programera, u jeziku C++ provjerava prevodilac.

Kasnije ćemo detaljno pričati o najvažnijoj novini koju uvodi C++, a to je **klasa.**

# C++ primjer

```cpp
#include <iostream>
#include <string>

int main() {
    std::string vlasnik;              // umjesto niza znakova char[30]
    std::cout << "Vlasnik: ";         // tok za izlaz, umjesto printf
    std::cin >> vlasnik;      // tok za ulaz (umjesto scanf), prihvata string do prve bjeline

    double uplate[3] = {120.5, 340.0, 75.25};

    auto ukupno = 0.0;          // kompajler sam izvodi tip double
    for (double iznos : uplate)    // petlja po opsegu umjesto indeksne petlje
        ukupno += iznos;


    std::cout << vlasnik << ": " << ukupno << " EUR\n";
}
```

Izlaz

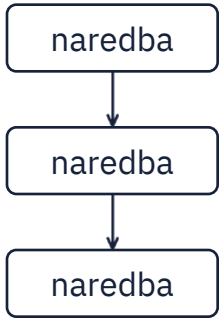
```
Vlasnik: Marko
Marko: 535.75 EUR
```

# Strukturirani/proceduralni program

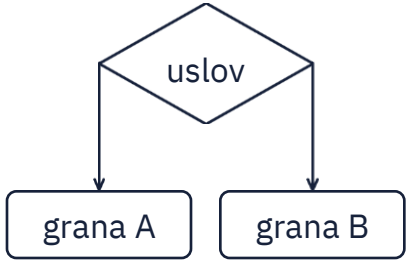
**kako teče izvršavanje**: sekvenca, grananje i petlja, bez proizvoljnih skokova (GOTO naredba)

**kako se program dijeli**: na funkcije sa parametrima i povratnom vrijednošću

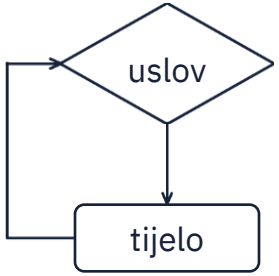
**Sekvenca**



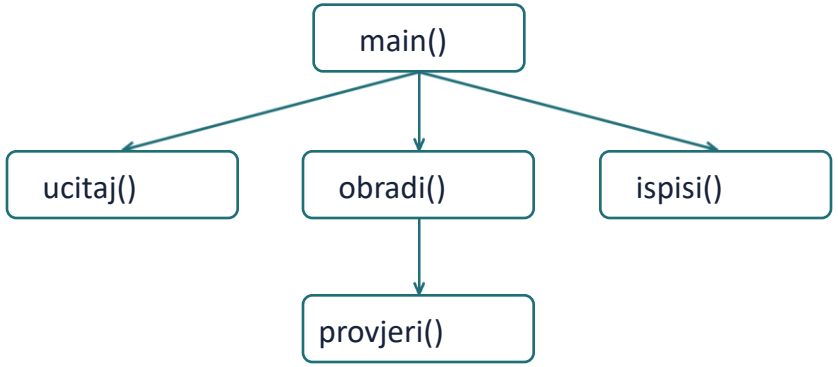

**Grananje**




**Petlja**




**Podjela na funkcije**




Iz ovih šema ne vidimo koji podaci pripadaju kojoj funkciji. **Zbog toga, sa proceduralnog/strukturiranog prelazimo na na Objektno-orijentisano programiranje**

# Ograničenja strukturiranog/proceduralnog programiranja

Strukturirano programiranje donijelo je niz prednosti, ali kako složenost softvera raste, nedostaci postaju ograničenje.

### Dvije velike cjeline

Program je podijeljen na **podatke** i na **kod (funkcije i procedure)**. Za složene programe ta podjela sama postaje problem.

### Novi tipovi podataka

Nedovoljno razvijena podrška za uvođenje novih sopstvenih tipova i proširenje postojećih.

### Integracija modula

Nedovoljno razvijeni mehanizmi za povezivanje modula koje pišu različiti timovi programera.

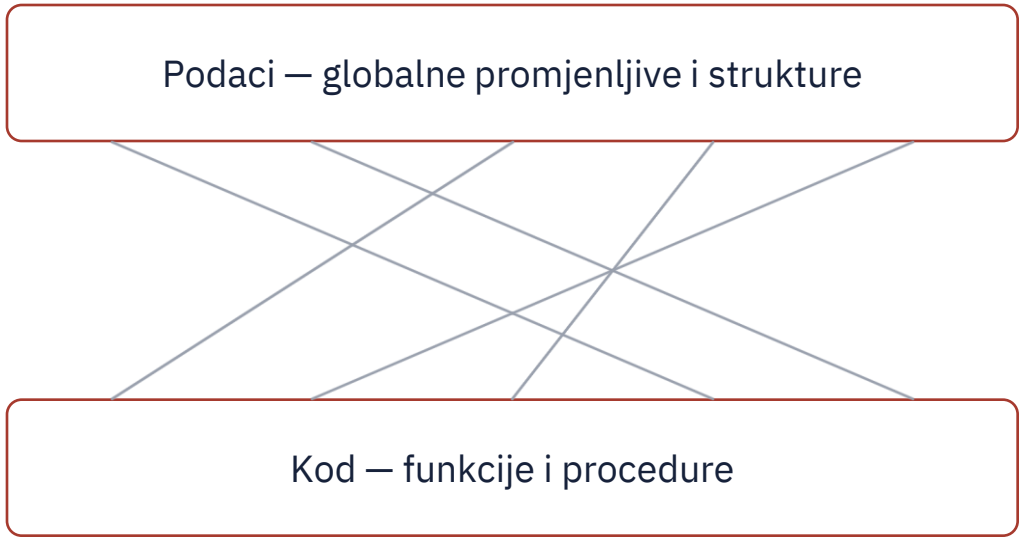
### Ponovna upotreba koda

Nedostatak mehanizama za ponovnu upotrebu jednom napisanog koda u drugom kontekstu.

> OO programiranje uvedeno je radi prevazilaženja navedenih nedostataka tradicionalnog strukturiranog/proceduralnog programiranja.

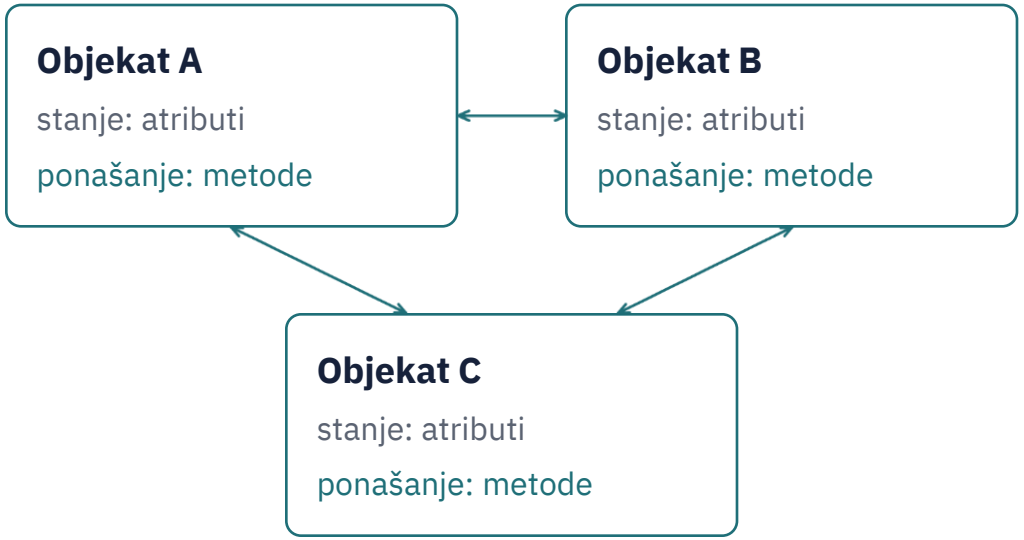
# Dva načina organizacije istog programa

## Proceduralna organizacija




Svaka funkcija smije pristupiti svakom podatku. Broj veza raste sa veličinom programa.

## Objektna organizacija




Podaci jednog entiteta dostupni su samo njegovim metodama. Veze postoje samo tamo gdje su potrebne.

# Entitet, stanje, ponašanje

- U objektnom pristupu softver se tretira kao model realnog svijeta.
- Svijet se sastoji od entiteta; entiteti imaju svoja stanja (atribute) i ponašanja (akcije, metode).

- **Klasa** je osnovni koncept koji predstavlja apstrakciju, odnosno model jednog entiteta iz domena problema.
  - npr. ako pravimo softver za bankarski sektor, vrlo vjerovatno imaćemo klase Filijala, Šalter, Zaposleni, Kamata itd.

- Podaci i kod vezani za jedan entitet čuvaju se u okviru jedne programske jedinice (jedne klase)

- Lakše je kontrolisati **više** manjih jedinica nego **dvije** velike, pa se i traženje grešaka pojednostavljuje.

| Entitet | Stanje | Ponašanje |
|---|---|---|
| Bankovni račun | vlasnik, stanje | uplata, isplata |
| Student | ime, indeks, ocjene | upis ocjene, prosjek |
| Rezervoar | kapacitet, količina | punjenje, pražnjenje |

# Zadatak: bankovni račun

Program vodi račun jednog klijenta: pamti ime vlasnika i stanje, prima uplate, izvršava isplate i ispisuje stanje.

# Verzija 0: Proceduralno rješenje - globalne promjenljive i funkcije

```
#include <iostream>
#include <string>

std::string vlasnik = "Marko Markovic";
double stanje  = 1000.0;
```

podaci

```
void uplata(double iznos)  { stanje += iznos; }
void isplata(double iznos) { stanje -= iznos; }

void prikazi() {
    std::cout << vlasnik << ": " << stanje << " EUR\n";
}
```

funkcije

```
int main() {
    prikazi();
    uplata(250);
    isplata(100);
    prikazi();
}
```

Izlaz

```
Marko Markovic: 1000 EUR
Marko Markovic: 1150 EUR
```

Podaci i funkcije razdvojeni su u dvije cjeline

PRVO OGRANIČENJE

# Sistem teba da podrži više računa

Dodajmo drugog klijenta, sa početnim stanjem 500 EUR

# Verzija 1: neodrživo rješenje za veći broj klijenata

```
#include <iostream>
#include <string>

std::string vlasnik = "Marko Markovic";
double stanje  = 1000.0;

std::string vlasnik1 = „Janko Jankovic";
double stanje1  = 500.0;

void uplata(double iznos)  { stanje += iznos; }
void isplata(double iznos) { stanje -= iznos; }

void uplata1(double iznos)  { stanje1 += iznos; }
void isplata1(double iznos) { stanje1 -= iznos; }

void prikazi() {
    std::cout << vlasnik << ": " << stanje << " EUR\n";
    std::cout << vlasnik1 << ": " << stanje1 << " EUR\n";

}
```

# Verzija 1.1: struct kao agregat podataka

```
struct Racun {
    string vlasnik;
    double stanje;
};

void uplata(Racun& r, double iznos)  { r.stanje += iznos; }
void isplata(Racun& r, double iznos) { r.stanje -= iznos; }

void prikazi(const Racun& r) {
    std::cout << r.vlasnik << ": " << r.stanje << " EUR\n";
}
int main() {
    Racun a{"Marko Markovic", 1000};
    Racun b{"Ana Anic", 500};

    uplata(a, 250);
    isplata(b, 100);

    prikazi(a);
    prikazi(b);
}
```

***struct* je koncept naslijeđen iz jezika C**

Globalnih promjenljivih više nema. Svaki račun je zasebna promjenljiva koja se predaje funkciji.

Funkcija radi nad bilo kojim računom.

Prenos referencom izbjegava kopiranje;
const ispred parametra označava da ga funkcija ne mijenja.

U mainu: dvije promjenljive istog tipa

Ovo je korektno rješenje u okvirima **strukturiranog** programiranja

Izlaz

```
Marko Markovic: 1250 EUR
Ana Anic: 400 EUR
```

**1. Dodatni uslov zadatka**

Stanje računa ni u jednom trenutku ne smije biti negativno.

**2. Dodatni uslov zadatka**

Uplata i isplata uvijek su strogo pozitivni iznosi.

**3. Dodatni uslov zadatka**

Nije moguće isplatiti iznos veći od raspoloživog stanja.

DRUGO OGRANIČENJE

## Ništa ne spriječava pozivaoca našeg koda da koristi naš kod na sljedeći način:

```
a.stanje = -5000;
isplata(b, 1000000); // stanje postaje -999500
```

Sva tri uslova iz postavke zadatka nisu osigurana kroz program

Ko je odgovoran? Autor *strukture Racun* ili autor funkcije *main*
Autor strukture je odgovoran

# Invarijanta tipa

DEFINICIJA

Invarijanta tipa je tvrdnja koja **mora biti tačna** prije i poslije svake operacije nad podatkom tog tipa. Ako je bilo koji dio programa može narušiti, ona nije invarijanta nego preporuka.

**Primjer: Bankovni račun**

stanje >= 0

**Primjer: Datum**

1 <= mjesec <= 12

# Provjera unutar funkcije

```
bool isplata(Racun& r, double iznos) {
    if (iznos <= 0 || iznos > r.stanje) return false;
    r.stanje -= iznos;
    return true;
}
```

**Uslov 3 je zaštićen**

```
isplata(b, 1000000);
// ako se funkcija pozove ovako, vratiće se false, stanje ostaje nepromijenjeno
```

**Uslov 1 i dalje nije zaštićen**

```
a.stanje = -5000;  // ova linija će proći bez problema, a to ne želimo
```

**Treba učiniti da se polju *stanje* ne smije pristupati izvan funkcija koje ga održavaju**

# Apstraktni tip podatka

DEFINICIJA

Apstrakcija je prepoznavanje bitnih svojstava sličnih objekata i njihovo opisivanje **novim tipom podatka**.

- Pored ugrađenih tipova, u objektnim jezicima programer može definisati i **sopstvene tipove podataka**.
- Sopstveni tipovi koriste se ravnopravno sa ugrađenim: moguće je kreirati proizvoljan broj promjenljivih tog tipa i definisati operacije nad njim.
- Sopstveni tip pravimo tako što definišemo **klasu**, a kreiranje **objekta neke klase** naziva se instanciranjem te klase.
- Objekat je instanca klase (konkretni primjer te klase)
- Termine **tip** i **klasa** koristimo kao sinonime

U jeziku C++ klasa se označava ključnom riječju **class**.

# Korak 1: struct postaje class

```
struct Racun {
    string vlasnik;
    double stanje;
};
```



```
class Racun {
    string vlasnik;
    double stanje;
};
```



```
class Racun {
  public:
    string vlasnik;
    double stanje;
};
```

```
void uplata(Racun& r, double iznos)  { r.stanje += iznos; }
void isplata(Racun& r, double iznos) { r.stanje -= iznos; }

void prikazi(const Racun& r) {
    std::cout << r.vlasnik << ": " << r.stanje << " EUR\n";
}
int main() {
    Racun a{"Marko Markovic", 1000};
    Racun b{"Ana Anic", 500};

    uplata(a, 250);
    isplata(b, 100);

    prikazi(a);
    prikazi(b);
}
```

U *klasi (class)* je sve **privatno** dok se ne specificira drugačije
U strukturi (*struct)* je sve javno dok se ne specificira drugačije
To je jedina razlika između ključnih riječi class i struct.

Bez oznake **public:** prevođenje ne bi uspjelo. Zašto?

Sa dodatom ključnom riječju **public,** program se prevodi i izvršava potpuno isto kao prethodna verzija.

# Korak 2: globalne funkcije postaju metode klase

```
class Racun {
public:
    std::string vlasnik;
    double stanje;

    void uplata(double iznos) { stanje += iznos; }

    bool isplata(double iznos) {
        if (iznos <= 0 || iznos > stanje) return false;
        stanje -= iznos;
        return true;
    }

    void prikazi() {
        std::cout << vlasnik << ": " << stanje << " EUR\n";
    }
};
```

parametar Racun& r nestaje kao i svi r. prefiksi

```
int main() {
    Racun a{"Marko Markovic", 1000};
    Racun b{"Ana Anic", 500};

    a.uplata(250);
    b.isplata(100);

    a.prikazi();
    b.prikazi();
}
```

> Poziv metode čita se kao rečenica prirodnog jezika: objekat a, izvrši uplatu iznosa 250.

## Korak 3: podaci postaju privatni

```
class Racun {
private:
    string vlasnik;
    double stanje;

public:
    void uplata(double iznos)  { ... }
    bool isplata(double iznos) { ... }
    void prikazi()             { ... }
};
```

**Podatku članu *stanje* više ne može da se pristupi van klase *Racun***

```
a.stanje = -5000;
error: 'double Racun::stanje' is
private within this context
```

**Greška koja je posljedica (klasa prestaje da bude agregat jer ima privatnih članova -> moramo definisati konstruktor klase sa dva argumenta)**

```
Racun a{"Marko Markovic", 1000};
error: no matching function for
call to 'Racun::Racun(...)'
```

Uslov „stanje računa nikada nije negativno“ od ovog trenutka ne zavisi od pažnje onoga ko koristi tip. Njegovo poštovanje provjerava prevodilac, pri svakom prevođenju.

# Na koji način se sada kreira račun?

**Dosadašnji način više nije moguć**

```
Racun a{"Marko Markovic", 1000};
```

Agregatna inicijalizacija upisivala je vrijednosti neposredno u atribute, a oni su sada privatni.

**Jedini preostali način**

```
Racun c; // kreiranje objekta klase Racun
c.prikazi();
```

```
: 6.95333e-310 EUR
```

Kreirali smo objekat sa neodređenim sadržajem memorije umjesto *stanja* — pri čemu ga više nije moguće ni ispraviti izvana. Imamo objekat sa neinicijalizovanom memorijom (kao kada napravite promjenljivu i odštampate vrijednost bez prethodne inicijalizacije)

# Konstruktor

```
class Racun {
private:
   string vlasnik;
   double stanje;

public:
   Racun(string ime, double pocetno) {
      vlasnik = ime;
      stanje  = pocetno;
   }

   // uplata, isplata i prikazi ostaju nepromijenjene
};
```

**Izmjena u funkciji main:**

```
// kreiranje dva objekta klase Racun; obične zagrade, umjesto vitičastih koje smo imali kod struct
Racun a("Marko Markovic", 1000);
Racun b("Ana Anic", 500);
```

**Od sada ćemo imati grešku pri prevođenju sljedeće linije:**

```
Racun c;
```

Čim klasa dobije sopstveni konstruktor, podrazumijevani konstruktor (bez parametara) prestaje da postoji – moramo ga definisati ako hoćemo da kreiramo objekte bez parametara

Konstruktor je metoda koja se poziva pri kreiranju objekta klase, nosi ime klase i nema povratni tip

Konstruktor se poziva automatski, tačno jednom, **u trenutku nastanka objekta**.

## Invarijantu integrišemo u konstruktor

```
Racun(string ime, double pocetno) {
    vlasnik = ime;
    stanje  = (pocetno > 0) ? pocetno : 0;
}
```

Račun sa negativnim stanjem od ovog trenutka ne postoji kao mogućnost.

# Enkapsulacija

DEFINICIJA

Implementacija tipa ostaje **skrivena** od ostatka sistema, dok je korisniku jasno definisano šta je sa tim tipom moguće raditi (šta taj tip nudi ostatku koda). Princip se sprovodi označavanjem pojedinih dijelova koda klase javnim ili privatnim.




```
a.stanje = -5000;
```

Direktan pristup privatnim podacima iz spoljašnjeg koda nije moguć. Svaki pristup ide kroz javni interfejs koji je autor klase omogucio

Analogija sa kućnim aparatom: korisnik vidi samo skup dugmadi putem kojih koristi aparat. Elektronika i mehanika koje izvršavaju operacije skrivene su od njega.

Efekat: smanjuje se mogućnost da dođe do greške, a traženje grešaka se pojednostavljuje jer je broj mjesta na kojima se podatak mijenja ograničen i poznat.

# Interfejs i implementacija

### Interfejs — šta klasa nudi ostatku koda

Interfejs ili javni dio klase čine zaglavlja metoda *uplata, isplata i prikazi*.
Ostatak programa je pravljen tako da se oslanja na trenutni interfejs naše klase, pa **interfejs treba mijenjati rijetko i sa pažnjom**.

### Implementacija — kako klasa implementira svoje metode

Privatni atributi i tijela metoda. Može se mijenjati slobodno, sve dok interfejs ostaje nepromijenjen.

# Primjer izmjene implementacije bez izmjene interfejsa

Napravili smo sljedeću klasu:

```
class Racun {
private:
    string vlasnik;
    double stanje;
public:
    Racun(string ime, double pocetno) {
        vlasnik = ime;
        stanje = (pocetno > 0) ? pocetno : 0;
    }
    void uplata(double iznos) { stanje += iznos; }
    bool isplata(double iznos) {
        if (iznos <= 0 || iznos > getStanje())
            return false;
        stanje -= iznos;
        return true;
    }
    double getStanje() const {
        return stanje;
    }
};
```

Ovo je kod koji koristi našu klasu:

```
int main ()
{

    Racun a("Marko Markovic", 1000);
    a.uplata(200);
    a.isplata(50);
    std::cout << a.getStanje() << std::endl;

    return 0;
}
```

Pretpostavimo sada da Banka želi da vidi ukupan promet, dakle koliko je bilo uplaćenog novca i koliko isplaćenog novca.

Umjesto jednog broja u atributu *stanje*, pamtićemo 2 broja: uplaćeno, isplaćeno.
Stanje ćemo dobiti kao razliku ta dva broja, kada god je stanje potrebno.

Ono što je najvažnije, korisnici klase (ostatak koda, u ovom slučaju funkcija *main*) neće osjetiti nikakvu razliku jer se interfejs ne mijenja

# Primjer izmjene implementacije bez izmjene interfejsa

**Prije · stanje se pamti kao atribut**

```
class Racun {
private:
   string vlasnik;
   double stanje;
public:
   Racun(string ime, double pocetno) {
      vlasnik = ime;
      stanje = (pocetno > 0) ? pocetno : 0;
   }
   void uplata(double iznos) { stanje += iznos; }
   bool isplata(double iznos) {
      if (iznos <= 0 || iznos > getStanje())
         return false;
      stanje -= iznos;
      return true;
   }
   double getStanje() const {
      return stanje;
   }
};
```

**Poslije · stanje se računa (kaže se i da je stanje izvedeni atribut klase)**

```
class Racun {
private:
   string vlasnik;
   double uplaceno;
   double isplaceno;
public:
   Racun(string ime, double pocetno) {
      vlasnik = ime;
      uplaceno = (pocetno > 0) ? pocetno : 0;
      isplaceno = 0;
   }
   void uplata(double iznos) { uplaceno += iznos; }
   bool isplata(double iznos) {
      if (iznos <= 0 || iznos > getStanje())
         return false;
      isplaceno += iznos;
      return true;
   }
   double getStanje() const {
      return uplaceno - isplaceno;
   }
};
```

Svi javni potpisi klase (javni interfejs) ostali su nepromijenjeni, pa se funkcija **main (niti bilo koji kod koji koristi našu klasu)** ne mijenja ni u jednom redu.

# Interfejs je ostao isti, a implementacija se promijenila

```
ostatak programa — funkcija main
a.uplata(200);  a.isplata(50);  a.getStanje();
```

**PRIJE**

**Javni dio — šta klasa nudi ostatku programa**

| | |
|---|---|
| Racun(string, double) | double getStanje() const |
| void uplata(double) | bool isplata(double) |

ispod ove linije ostatak programa ne vidi ništa

**Privatni dio — prije**

| | |
|---|---|
| string vlasnik | double stanje |

**POSLIJE**

**Javni dio — šta klasa nudi ostatku programa — ostaje isto**

| | |
|---|---|
| Racun(string, double) | double getStanje() const |
| void uplata(double) | bool isplata(double) |

ispod ove linije ostatak programa ne vidi ništa

**Privatni dio — poslije**

| | |
|---|---|
| string vlasnik | double uplaceno |
| double isplaceno | |

Atribut **stanje** u drugoj verziji ne postoji — računa se. Ostatak programa to ne može primijetiti.

# Dopišimo ovu funkciju izvan klase

```
void izvjestaj(const Racun& r) {
    r.prikazi();
}
```

```
error: passing 'const Racun'
as 'this' argument discards
qualifiers
```

Parametrom const obećano je da se objekat r neće mijenjati unutar funkcije izvjestaj, a zatim je pozvana metoda prikazi za koju prevodilac nema nikakav dokaz da ništa ne mijenja.

# Konstantne metode

```
void prikazi() const {
    cout << vlasnik << ": " << stanje << " EUR\n";
}

double getStanje() const { return stanje; }

string getVlasnik() const { return vlasnik; }
```

Oznaka const iza zaglavlja znači da metoda „obećava“ da neće mijenjati nijedan atribut objekta.

**Prevodilac će u ovom slučaju prijaviti grešku jer je metoda označena sa *const* pokušala da promijeni atribut objekta**

```
double getStanje() const {
    stanje = 0;  // error assignment of member in read-only object
    return stanje;
}
```

Preporuka: svaku metodu koja samo čita stanje objekta označiti sa const.